

硬核拆解 OpenClaw：如何构建真正稳健的生产级 Agent 系统？【上】

mp.weixin.qq.com/s/wwzUiY7NCeiFAhTss0Qq-w

秋山墨客

Original

尽管伴随着一些质疑，OpenClaw（原 Clawdbot）热度还在持续走高。不过，外行看热闹，内行看门道。面对这一现象级工具，若仅停留在“体验”层面，未免可惜。今天我们将掀开 OpenClaw 的“引擎盖”，深入剖析其最核心的动力模块 — Agent 任务引擎。即便 OpenClaw 大量借助 AI 编码，其架构上仍蕴含着宝贵的工程智慧。深刻理解这些实践，吸收甚至沉淀为明确的规范，有助于我们更精准的指导 AI，构建出真正稳健的生产级 Agent 系统，而不是上线就“破碎”的玩具。

由于篇幅较长，我们分成上下两篇。本篇首先来认识：

- OpenClaw Agent 的“工作室”
- OpenClaw Agent 的整体架构
- 调度与并发控制
- 高可用与容错机制

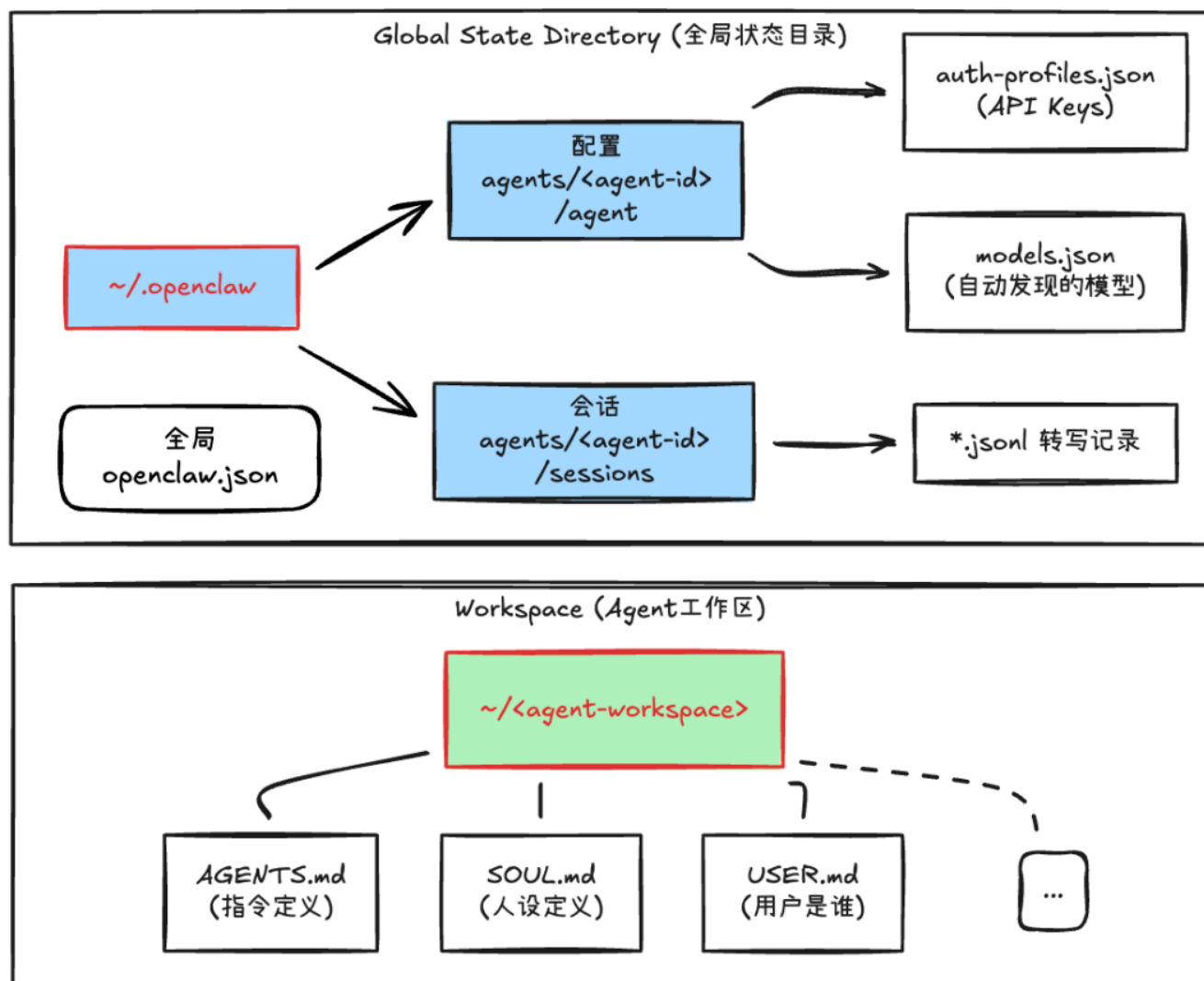


01

Agent 的“工作室”

你可以把 OpenClaw 的 Agent 模块看做一个持久化、有状态的“数字员工”。它不仅有大脑（LLM），还有配套的“办公环境” — 用来长期保存信息与产出物。

用一个简单的图来说明：



工作区 (Workspace) : Agent 的“办公桌”，保存必要档案与产出

位置：在创建 Agent 时可以自由指定本机目录

用途：类似 Agent 的“Git库”，存放AGENTS.md（行为指令）、SOUL.md（性格设定）等

配置 (Config) : Agent 使用的、敏感的运行时配置

位置：~/.openclaw/agents/<agent-id>/agent (agent-id 为Agent名称)

用途：存放 Agent 系统级配置，比如敏感凭证、可用的模型等

会话 (Sessions) : 记录 Agent 和你“聊过什么，干过什么”

位置：~/.openclaw/agents/<agent-id>/sessions

用途：存放 Agent 的对话历史(.jsonl)，属于 State 的一部分

由于 OpenClaw 支持添加多个 Agent，可以看到每个 Agent 都会有自己独立的工作区与状态区，互不干扰。

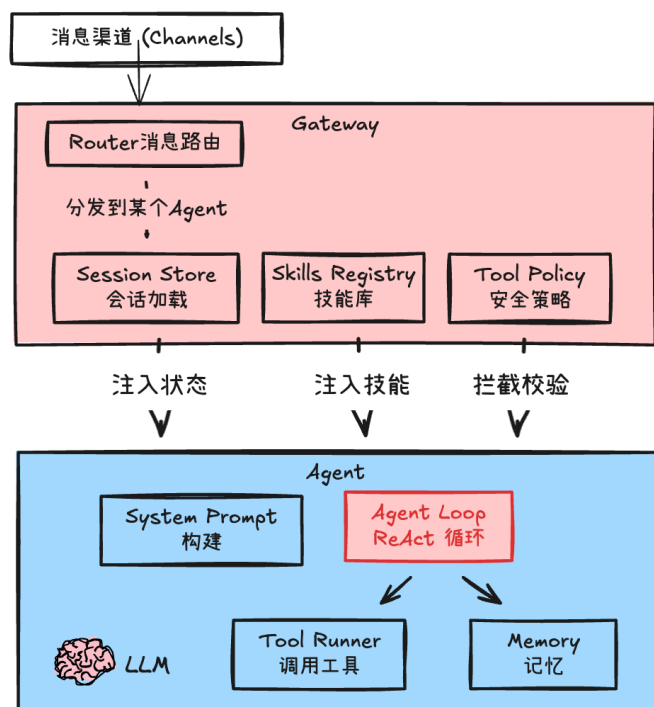
【关键解读】

这里的关键是把 Agent 用户侧 (Workspace) 与系统侧 (State) 数据的分离 — 用户侧可以分发与版本控制（方便把 Agent 复制给别人或生产环境）；而系统侧则保持私密，API Key、会话历史等，确保不会因为分发或迁移而被泄露。

Agent 的整体架构

在OpenClaw中，Agent 不是一个直接由客户端驱动的任务模块。它由OpenClaw 的指挥中枢 Gateway 统一进行接入与管控。

整体架构如下：



各主要模块分工：

- **Channels**：对接外部平台（WhatsApp等），负责消息接入与适配
- **Gateway**：管理与指挥中心，负责将消息路由到某个 Agent、加载 Agent 会话、给 Agent 注入 Skills 索引、给 Agent 配备与注入可用工具
- **Agent**：动态构建上下文（时间、偏好、技能、工具等），并驱动 "思考 -> 工具 -> 结果 -> 思考" 的 ReAct 循环（依赖LLM）
- **LLM**：负责推理的大脑。Agent 把推理上下文交给模型，模型输出下一步行动或者结果。

【关键解读】

通过 **Gateway + Agent**的分层，把“接入与管控”和“业务执行”拆开，这在企业级 Agent 落地中很有参考价值：

- **统一接入**：全渠道接入、消息适配、路由，进一步承载统一认证/身份体系
- **统一安全**：Gateway 会对高风险工具做把关与物理“拦截”（后文细讲）。好处是：即使 Agent误判或被“注入”，也无法越过策略执行高危工具。

并发与资源控制：可以在 Gateway 侧统一做限流、排队、超时等治理策略

具体到真实场景，当你有多个 Agent、多个渠道、多个工具时，可以参考这里的架构实践，比如构建企业统一的 AI 中台、需要高安全级的运维机器人、大规模智能客服等。

当然，同时也要看到代价：Gateway 会成为系统关键路径，设计不好容易变“瓶颈/单点”，需要配套可观测性与容灾能力。

调度与并发控制

真实场景下，对话式的Agent 会话过程往往很复杂。由于它有状态、且可能长时运行，用户还会持续输入新消息，你不能像无状态 API 那样简单的把所有消息（任务）并发处理，否则很容易导致混乱与 token 浪费。

“车道”隔离：消除任务竞态

如果你连续发这样的消息：

“帮我查下天气” → “不是，查下股市行情” → “算了还是查天气吧”。

如果为这三条消息启动三个并行的 Agent 处理，同时读写 Session 状态，容易导致状态损坏（如相互覆盖）和上下文错乱 — 进而导致模型推理的错误。

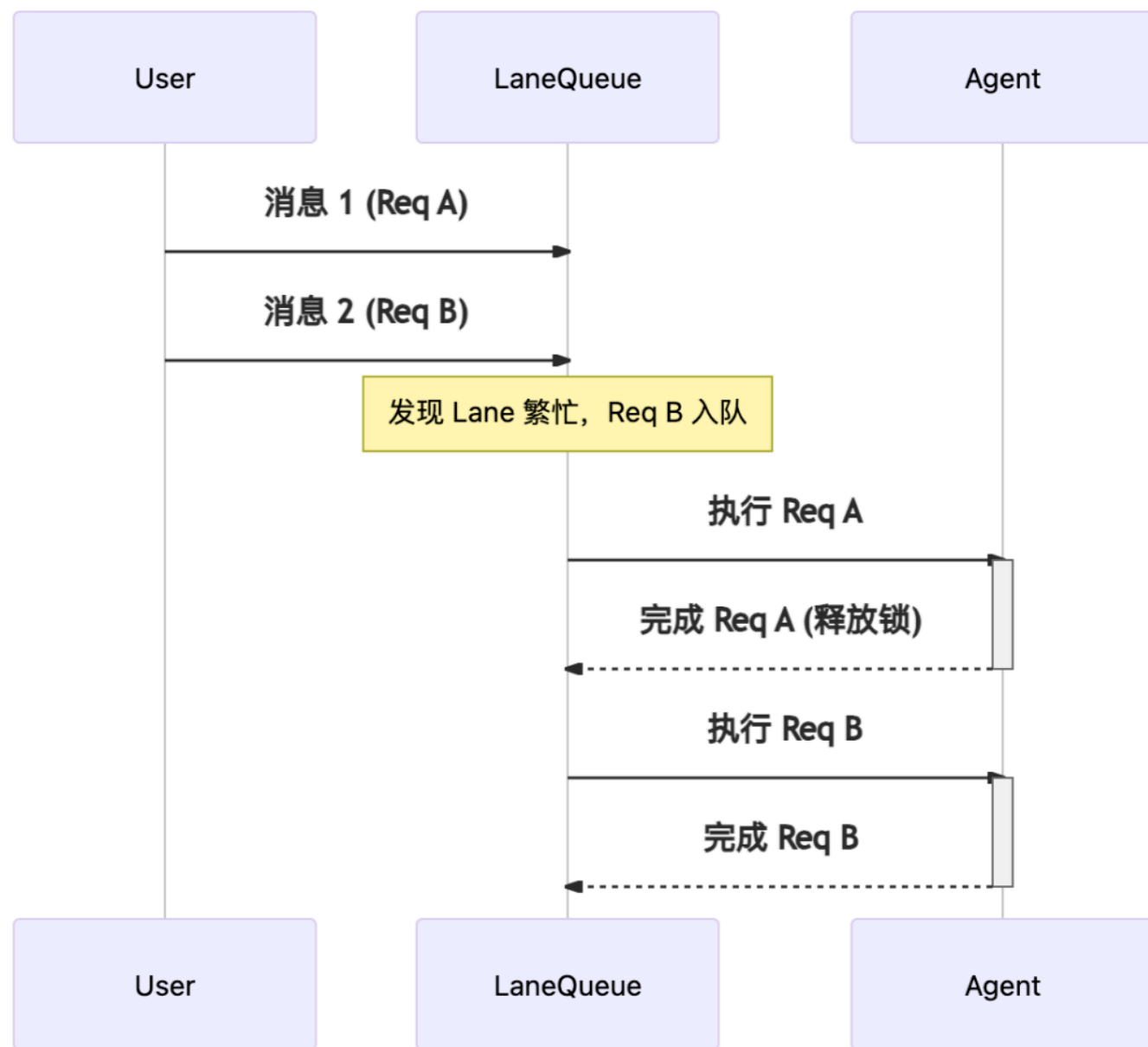
所以，需要一套机制来保证消息（任务）的处理时序正确。

【OpenClaw 的方法】

OpenClaw 的做法是：为每个 Session 分配唯一的 Lane（“车道”），对应一个内存中的串行队列。当新消息到达时，如果该“车道”正在忙（默认的最大并发数为1），就进入队列等待。

```
type LaneState = {  
  lane: string;  
  queue: QueueEntry[]; // 待处理的消息队列  
  active: number; // 当前并发数，与maxConcurrent配合，但通常为1  
  draining: boolean; // 是否正在调度中  
};
```

以两个连续的消息为例，调度过程如下：



【关键解读】

对**有状态 Agent**来说，“同一会话任务串行化”是保证一致性的低成本方案 — 不要轻易并发同一用户的多轮对话，尤其当工作流还涉及写操作甚至人类参与时。

当然，串行队列会牺牲单会话吞吐，遇到慢任务容易“堵车”。企业落地时通常可以配套**超时控制**、**优先级策略**、**异步**等机制，避免一个慢请求影响体验。后面还会分析到这方面的实践。

多种队列模式：应对高频输入

光有任务的排队还不够。OpenClaw 的入口来自 IM 渠道，很容易出现碎片化的连续输入，比如，这几个连续消息：

“你好”，“我想问下”，“如何给 OpenClaw 安装新的 Skill”，“能帮我安装吗”

如果这里的每条消息都触发一次 Agent，不仅浪费 token，而且没有意义 — 本质上它只是一个任务，这会显得 Agent 很傻。

【OpenClaw 的方法】

OpenClaw 允许你配置多种策略：用来在进入任务车道（Lane）之前，控制高频输入消息。常见的几种策略如下：

Steer (转向) 模式：在一个时间窗（比如2s）内的新消息会被“注入”到正在运行的上一轮 Agent Loop；如果失败，则等待下一轮Loop。场景：

“查询下南京天气” → “记得要最近一周的”

如何“注入”？

目前的方式是在Agent的ReAct循环中的某一次LLM调用结束后的间隙，被追加到消息流，从而参与到下一轮推理中。

- **Collect (搜集) 模式**：在时间窗内先等待，把窗口内到达的消息聚合成一条，再作为任务交给 Agent。场景：

“你好”“在吗”“帮我研究下这只股票行情”“代码是 xxxx”

- **Followup (跟进) 模式**：按 FIFO 顺序逐条处理，每条消息都作为一次 Agent 任务获得一次完整回复。场景：

“先处理任务1：xxx” → “再处理任务2：xxx”

- **Interrupt (打断) 模式**：时间窗内的新消息到来后直接中断当前运行、清空队列并丢弃回复，从新消息重新开始。场景：

“预定今天的会议室” → “不要做了，定明天的会议室”

这里的高频消息队列，要与前面的Lane队列区分。我们用“餐厅点餐”来比喻：

- **高频消息队列 = 服务员**，客人不断喊话，服务员决定怎么处理：
 - steer**：客人紧急修改，赶紧通知到厨房（比如“少放辣”）
 - collect**：确定客人说完“再来个...还要个...”，一次性下单
 - followup**：客人每点一道菜，就送厨房一次
 - interrupt**：之前的全部撤单，只做最新这道
- **Lane车道 = 厨房灶台**：不管服务员怎么整理，灶台同一时间只炒一盘菜

【关键解读】

在 OpenClaw 里，一次任务的成本不低（token 消耗 + 运行时间）。因此，把消息缓冲/聚合前置到 Agent 之前（Gateway 层）能明显提升体验，也能减少不必要的调用开销 — 这对大多数对话式 Agent 都适用。

04

高可用与容错机制

生产级系统和原型的关键差异，在于环境更复杂、也更不可预测。很多时候，让系统**足够健壮、可容错**，比“功能更强、UI漂亮”更重要。

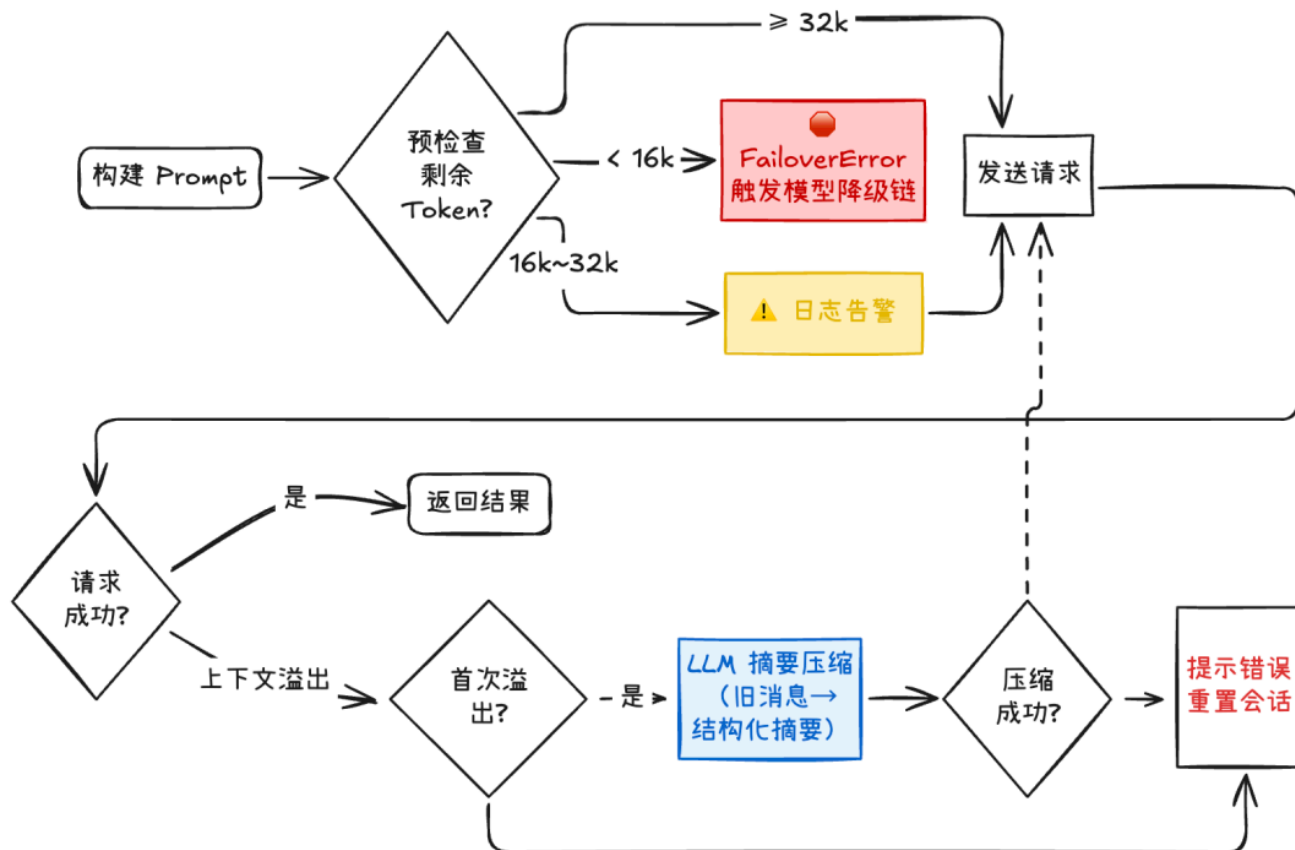
上下文守卫机制

在 LLM 应用中，上下文窗口是有限资源（例如 128k）。对多轮对话的 Agent 来说，随着会话变长，再叠加知识与工具定义，Prompt 很容易超限。如果只是让 LLM 抛出“上下文长度超限”，不仅浪费请求时间，还会让服务直接不可用。

【OpenClaw的方法】

OpenClaw实现了Context Guard（上下文守卫）的机制，构建了多阶段防线：

1. Prompt 构建完成但尚未调用 LLM 前：按阈值做 **token 预检**
2. **通过了预检，但运行过程中仍触发上下文溢出**：启动自动压缩
3. 压缩失败：此系统会尝试重置会话，并提示用户



注意这里的压缩并非简单的消息截断，而是：反向遍历消息，保留一部分最近原始消息；将更早内容用 LLM 压缩为结构化摘要；再拼接成新的输入。

另外，在 OpenClaw 使用的 SDK 内部还有一道“自动压缩”：当 token 使用量达到阈值，就提前触发压缩，而不是等 LLM 报错。

【关键解读】

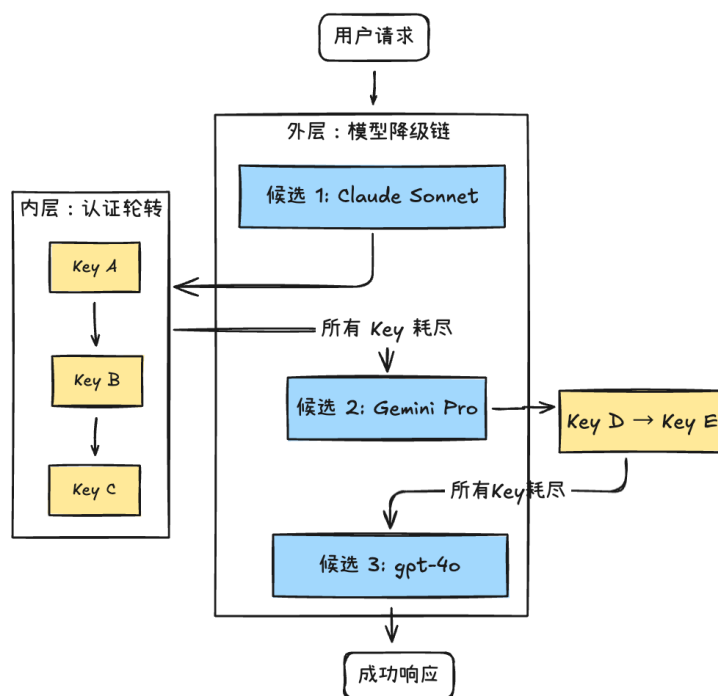
成熟的 Agent 系统要用“**事前检查 + 事后补救**”的多层策略，尽量别让 LLM 报错成为用户看到的最终错误。这本质上属于**上下文工程**：应用层必须掌控“保留什么、压缩什么、何时降级”。当然上下文的压缩是有代价的：会带来**信息损失与摘要偏差**，而重置会话也可能影响连续性。企业场景里可以根据需要配置独特的摘要策略、关键事实保护（如订单号/金额/日期）、以及可追溯的日志等，尽量避免压缩后的跑偏。

模型故障容错

在生产环境中，你需要假定大模型 API 不稳定是常态：网络、限流、余额/配额耗尽，甚至供应商整体故障。单一 Key + 单一模型会让系统非常脆弱 — 一个 Key 被限流，就可能导致 Agent 不可用。

【OpenClaw 的方法】

OpenClaw在两个层级上实现对 LLM 调用的容错：内层认证轮转 + 外层模型降级。原理很简单：



认证轮转：同一个模型 Provider 内，多个 API Key 实现自动切换 — 不断尝试下一个不在冷却（指数退避机制）期内的 Key，直到成功或所有 Key 耗尽

模型降级：当某个 Provider 的所有 Key 都失败后，自动降级到下一个候选模型。

【关键解读】

生产级 Agent 应把 LLM 当作**不可靠的外部依赖**（无论云端还是本地），像微服务对待下游服务一样设计：内层缓解同一模型 Provider 的短期波动，外层应对 Provider 的直接不可用。这种多重保护能让单点 LLM 故障更难把服务“击穿”。

不过，模型降级可能带来**能力差异与风格漂移**，甚至影响工具调用稳定性。企业落地时往往需要明确**模型分层与各自能力基线**（哪些任务允许降级、哪些必须强模型、允许降级到哪个模型等），并记录降级日志，便于排查与审计。

我们将在【下篇】继续了解 OpenClaw 在安全与风险防御、长期记忆、模型成本优化、子 Agent 协作机制等方面的工程实践。欢迎继续关注！